

Notite MAP- 2023

Anchei

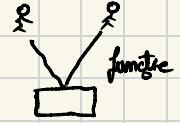
Cosmino



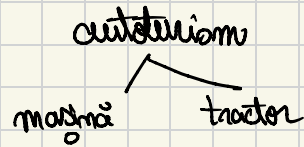
Introducere în JAVA

Concepte OOP

- abstractizarea = "ascundem" anumite funcționalități pe care nu e nevoie să le ști
- încapsularea = ținem datele și funcțiile separate de exterior
- moștenirea = dă voce unei clase să poată prop din altele clase
- polimorfism = putem crea mai multe copii ale aceluși metode
- clasă = un tip de dată
- obiect = o instanță a unei clase
- mesaj = felicit de date pt. a comunica



ex: evitare suprascriere în AB



ex: adăugăm la mașină și VIN

Noțiuni introductive

- `System.out.println();` → print on screen
↳ System command ↳ print line ↳ output
- declarare variabile
 - la fel ca în C++
 - `String driver; driver = "Jon";`

• String things

- `str.length()`; : dimensiune `str`
- `str = str.toUpperCase()`; `Jon` $\Rightarrow$ `JOH`
- `str m = str1 + str2`; `Jon, Popescu` $\Rightarrow$ `JonPopescu`
 $\hookrightarrow$ se poate face `str + " " +`

• Tipuri de date

- ca cele din C++
- void NU este tip de date

- byte: 1 octet (8 bits)

• Operatori

- aritmetici: `*`, `/`, `%`, `+`, `-`
- relaționali: `<`, `>`, `<=`, `>=`, `==`, `!=`
- logici: `||`, `&&`, `^`, `&`, `!` $\rightarrow$ OR logic
 $\hookrightarrow$ XOR $\hookrightarrow$ AND logic

! `double future = c * r;`
`int approx = (int) future;`

- ternari: `?` $\rightarrow$ `rez = (m % 2 == 0) ? "Par" : "Impar";`
- accesare, apelare: `.`, `[]`, `()`
- inc/dec: `expr++`, `expr--`
- Shift: `<<`, `>>`, `>>>`
 $\hookrightarrow$ ca `>>`, dar pe numere fără semn

• variabile și constante

- final `int max = 100;`
 $\hookrightarrow$ constantă
- `var list = new ArrayList<String>();`
 $\hookrightarrow$ variabilă

• tablouri unidimensionale

- tip `IS` `nume_tablou`; // declarație
- `nume_tablou = new tip [dim];`

- vect. length; // dim. $n \times 2$
- tablouri N -dimensionale
 - int $i \leq a_i$

Class in Java

```
[public] [abstract] [final] class NumarClaz {
    ...
}
```

- ! într-un fișier .java, doar o clasă poate fi publică
- ! final ; la sf. clasei

→ public: membru accesibil tuturor
 → private: membru accesibil doar clasei
 → protected: membru accesibil doar clasei și subclasei

- this. real = real; // this - referință la obiectul curent

→ moștenirea claselor

```
class Subclasa extends SupraClasa {
    ...
}
```

Interfață

- sunt implementate de clase
- conține doar metode abstracte, default și statice.

Genericitate

- cast - convertirea valorilor dintr-un tip de date în altul

ex: `int value = 10;`
`double dValue = int value;`

`double dValue = 10.5;`
`int intValue = (int) dValue;`

e ok pt că `int < double`

- tipuri genice - ca în C++ (`public class Node<T>..`)
↳ E - element, K - key, N - number, T - type, V - value

Singleton

→ permite restricționarea numărului de instanțieri ale unei clase la un singur obiect

- ex: pt. obține tip Factory.

- 2 abordări → Lazy Object instantiation

↓
Eager object instantiation

obiectul este înscris în memorie în cazul în care poate va fi folosit

obiectul este înscris în memorie doar dacă este folosit.

`if (instance == null)`

Excepție

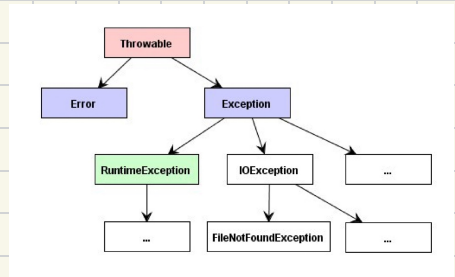
try {

```
} catch (TipExcepție e) {
    System.out.println(e);
}
```

- throw TipExcepție

- putem face propriile clase de excepție

ex: public class ExcepțieProprie extends RuntimeException {
 ...
}



Stream → orice oasă sau consumator de date care este capabil să producă sau să primească unități de date

```
abstract int read()
    Reads the next byte of data from the input stream.

int read(byte[] b)
    Reads some number of bytes from the input stream and stores them into the buffer array b.

int read(byte[] b, int off, int len)
    Reads up to len bytes of data from the input stream into an array of bytes.

void reset()
    Repositions this stream to the position at the time the mark method was last called on this input stream.

long skip(long n)
    Skips over and discards n bytes of data from this input stream.
```

```
void close()
    Closes this output stream and releases any system resources associated with this stream.

void flush()
    Flushes this output stream and forces any buffered output bytes to be written out.

void write(byte[] b)
    Writes b.length bytes from the specified byte array to this output stream.

void write(byte[] b, int off, int len)
    Writes len bytes from the specified byte array starting at offset off to this output stream.

abstract void write(int b)
    Writes the specified byte to this output stream.
```

- File InputStream / File OutputStream

JAVA 8

Interfete funcționale

- conține doar o metodă abstractă

```
@FunctionalInterface  
interface Operatie {
```

```
    int adunare (int a, int b);  
}
```

- permite crearea de instanțe folosind expresii lambda
- pot conține și metode default / statice

Metode statice

- nu depind de o instanță a unei clase
- sunt denumite direct prin numele interfeței
- sunt apelate folosind numele interfeței

```
ex: interface Utilitati {
```

```
    static void afiseaza (String msg) {  
        System.out.println (msg);  
    }
```

```
Utilitati.afiseaza ("Mesaj");
```

Metode predefinite

- implementare implicită pt interfață
- pot fi suprascrise de clasele ce implementează interfața

ex: interface List<T> {

```
    default void afis() {  
        System.out.println("mesaj" + size());  
    }  
}
```

class Lista implements List<String> {

```
    @Override  
    public int size() {  
        return 5;  
    }  
}
```

main:

```
Lista lista = new Lista();  
lista.afis();
```

Funcții lambda

↳ funcție anonimă

- nu este legată de un identificator

ex: (int a, int b) → a + b

- simplificare expresii / înlocuire blocuri de cod
- folosite în operațiile pe stream (filtrare, mapare, reducere)

ex: `List<String> lista = Array.asList("A", "B");`
`lista.forEach(System.out::println);`

Metode referință

Referința Obiect :: nume metoda

Interfețe funcționale din Java

- Predicate → o condiție sau un test asupra unui obiect
→ `test(T t)`
- Consumer → o acțiune sau o operație asupra unui obiect
- Function → primește date de intrare și returnează un rez.
→ `R apply(T t)`;
- Supplier → furnizor de valori, nu primește argumente
→ `T get()`;
- Comparator → utilizat în sortarea elementelor
→ returnează un rezultat negativ, zero sau pozitiv
→ `int compare(T o1, T o2)`;
- Optional → gestionare situațiilor în care o val. poate lipsi sau poate fi prezentă
→ evita valorile null

Stream → permite operațiuni de procesare a datelor într-un mod elegant și funcțional

→ operațiuni: filter, map → transf. elem. în altă valoare,
distinct → elim. duplicatele, sorted,
limit → limitare m. elem., skip

→ operațiuni terminale: `forEach`, `toArray`, `collect`, `reduce`,
`min`, `max`, `count`, `findAny`, `match`

Java FX

- clase și interfețe ce oferă suport de creare aplicații Java
ce se pot proiecta/implementa pe diverse platforme
- interfețe grafice + CSS

Java FX

- `window = stage`
↳ `content = scene`
- `stage` • — `setTitle`

`StackPane` - layout

`layout.getChildren()`
• `add()`

`stage` → `layout` → `scene`